

01 – Databases & Entity-Relationship Diagram

From the real-world to a computer model of your database

Jérôme Landré – jerome.landre@ju.se

OUTLINE

- 1) Data & databases
- 2) The Relational Databases
- 3) The Entity-Relationship (ER) Model
 - 3.1. The conceptual model
 - 3.2. The logical model
 - 3.3. The physical model
- 4) MS Structured Query Language (SQL)
- 5) MariaDB SQL
- 6) Standard relationships
 - 6.1) One-to-One relationships
 - 6.2) One-to-Many/Many-to-One relationships
 - 6.3) Many-To-Many relationships
- 7) Special relationships
 - 7.1) Generalization
 - 7.2) Specialization
 - 7.3) Inheritance

1) DATA & DATABASES

- ” In common usage, **data** is a **collection of** discrete or continuous **values** that **convey information**, describing the quantity, quality, fact, statistics, other **basic units of meaning**, or simply sequences of symbols that may be further interpreted formally. ” Source: <https://en.wikipedia.org/wiki/Data>
- Data about me:
 - First name, Last name, Birth place, Date of birth, Age
 - Address, Number, Street, Zip code, Country...
- Data about a car:
 - Brand, Model, Color, Engine power, Energy, Kilometers, Year...

1) DATA & DATABASES

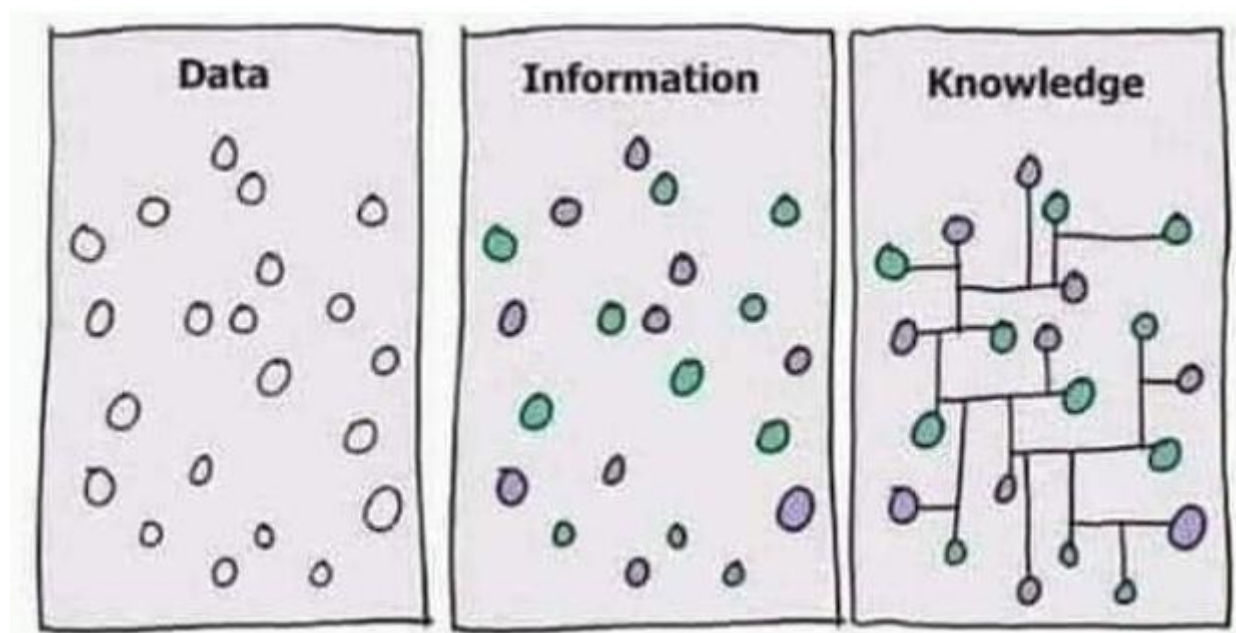
- ” In computing, a **database** is an **organized collection of data** or a type of data store based on the use of a **database management system (DBMS)**, the software that **interacts** with end **users**, **applications**, and the database itself to capture and analyze the data. ” Source: <https://en.wikipedia.org/wiki/Database>
- There are many DataBase Management Systems (DBMS)

1) DATA & DATABASES

- There are many models for databases:
 - **Navigational** DBMS (1960s)
 - **Relational** DBMS on "big" computers (1970s)
 - **Relational** DBMS on Personal Computer (1980s)
 - **Object Databases** and Object-relational Databases (1990s)
 - **NoSQL** and **NewSQL** (2000s)
- The classification of databases is complicated, it depends on many parameters. Source: <https://en.wikipedia.org/wiki/Database>

1) DATA & DATABASES

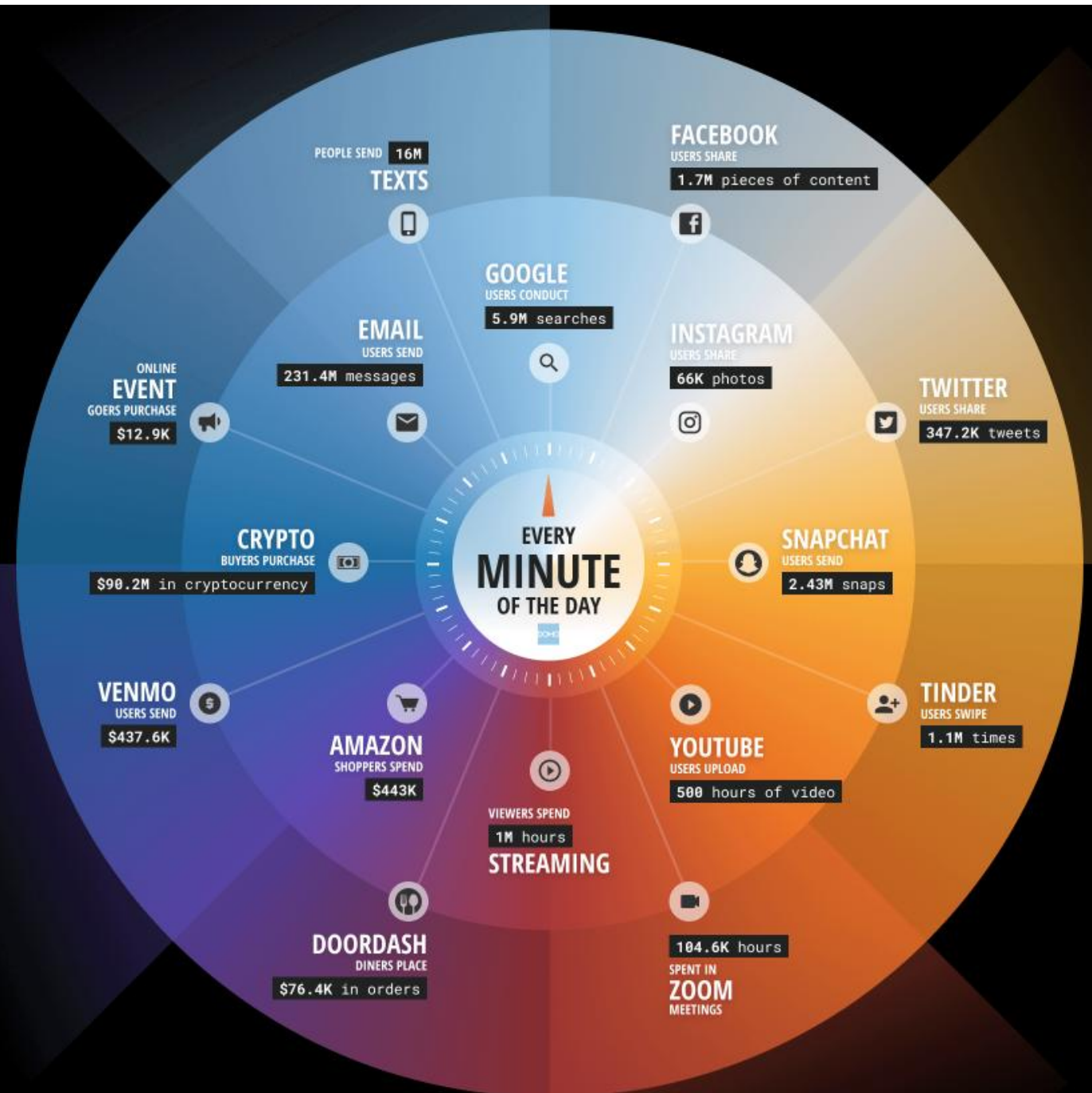
- Difference between Data, Information, and Knowledge... Source: <https://www.linkedin.com/pulse/data-information-knowledge-insights-achin-gupta/>



1) DATA & DATABASES

- Data everywhere...
- Data creation every minute of the day...

Source: <https://www.domo.com/data-never-sleeps>



1) DATA & DATABASES

- Data everywhere...
- Data creation every minute of the day...

Source: <https://www.domo.com/data-never-sleeps>



If you have the data, you have the power!
Data is Money!

2) THE RELATIONAL DATABASES

- ” In a **relational database**, a relation is a set of tuples that have the same attributes. A tuple usually represents an object and information about that object. Objects are typically physical objects or concepts. A relation is usually described as a **table**, which is organized into **rows and columns**. All the data referenced by an **attribute** are in the same domain and conform to the same constraints. “

Source: https://en.wikipedia.org/wiki/Relational_database

2) THE RELATIONAL DATABASES

- The Relational Databases (RDB) use:
 - **Entities** (**tables**) containing data organized in rows and columns
 - Columns are the **attributes** or **fields**
 - Rows are the **tuples** or **records**
 - One special column must be the **primary key (PK)** and has a **unique value** for each tuple in the table

File Edit View Query Project Tools Window Help



Object Explorer

Connect

- External Tables
- Graph Tables
- dbo.AWBuildVersion
- dbo.DatabaseLog
- dbo.ErrorLog
- HumanResources.Department
- HumanResources.Employee
- HumanResources.EmployeeD
- HumanResources.EmployeeP
- HumanResources.JobCandid
- HumanResources.Shift
- Person.Address
- Person.AddressType
- Person.BusinessEntity
- Person.BusinessEntityAddress
- Person.BusinessEntityContact
- Person.ContactType
- Person.CountryRegion
- Person.EmailAddress
- Person.Password
- Person.Person
- Person.PersonPhone
- Person.PhoneNumberType
- Person.StateProvince
- Production.BillOfMaterials
- Production.Culture
- Production.Document
- Production.Illustration
- Production.Location
- Production.Product
- Production.ProductCategory
- Production.ProductCostHist

SQLQuery1.sql - jth...019 (AD\lanjer (69))

```
SELECT TOP (1000) [BusinessEntityID]
, [PersonType]
, [NameStyle]
, [Title]
, [FirstName]
, [MiddleName]
, [LastName]
, [Suffix]
, [EmailPromotion]
, [AdditionalContactInfo]
, [Demographics]
, [rowguid]
, [ModifiedDate]
FROM [AdventureWorks2019].[Person].[Person]
```

110 %

Results Messages

	BusinessEntityID	PersonType	NameStyle	Title	FirstName	MiddleName	LastName	Suffix	EmailPromotion	AdditionalContactInfo	Demographics
1	1	EM	0	NULL	Ken	J	Sánchez	NULL	0	NULL	<IndividualSurvey xmlns="http://schemas.microso.
2	2	EM	0	NULL	Teri	Lee	Duffy	NULL	1	NULL	<IndividualSurvey xmlns="http://schemas.microso.
3	3	EM	0	NULL	Roberto	NULL	Tamburello	NULL	0	NULL	<IndividualSurvey xmlns="http://schemas.microso.
4	4	EM	0	NULL	Rob	NULL	Walters	NULL	0	NULL	<IndividualSurvey xmlns="http://schemas.microso.
5	5	EM	0	Ms.	Gail	A	Erickson	NULL	0	NULL	<IndividualSurvey xmlns="http://schemas.microso.
6	6	EM	0	Mr.	Jossef	H	Goldberg	NULL	0	NULL	<IndividualSurvey xmlns="http://schemas.microso.
7	7	EM	0	NULL	Dylan	A	Miller	NULL	2	NULL	<IndividualSurvey xmlns="http://schemas.microso.
8	8	EM	0	NULL	Diane	L	Margheim	NULL	0	NULL	<IndividualSurvey xmlns="http://schemas.microso.
9	9	EM	0	NULL	Gigi	N	Matthew	NULL	0	NULL	<IndividualSurvey xmlns="http://schemas.microso.
10	10	EM	0	NULL	Michael	NULL	Raheem	NULL	2	NULL	<IndividualSurvey xmlns="http://schemas.microso.
11	11	EM	0	NULL	Ovidiu	V	Cracium	NULL	0	NULL	<IndividualSurvey xmlns="http://schemas.microso.
12	12	EM	0	NULL	Thierry	B	D'Hers	NULL	2	NULL	<IndividualSurvey xmlns="http://schemas.microso.

Query executed successfully.

jthstuds\sql.win.hj.se (15.0 ... | AD\lanjer (69) | AdventureWorks2019 | 00:00:00 | 1 000 rows

TIPS AND TRICKS

- Some **French** words...

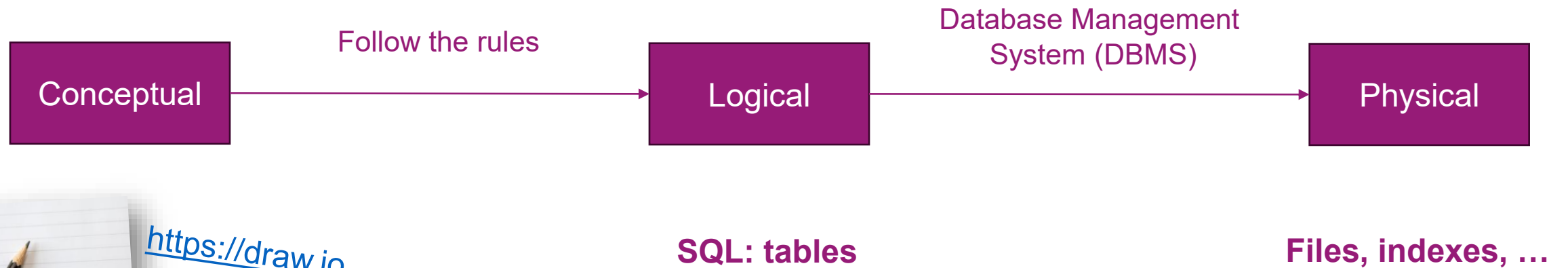
Bonjour	Good day	God dag	formal/informal first greeting to a person or a group of persons
Au revoir	Good bye	Hej då	formal/informal last greeting to a person or a group of persons
Ça va ?	How are you?	Hur mår du?	Informal way to ask persons how they feel.
Ça va.	I'm fine.	Jag mår bra.	Informal way to answer that you are well.
Oui	Yes	Ja	formal/informal way to agree
Non	No	Nej	formal/informal way to disagree

3) THE ENTITY-RELATIONSHIP (ER) MODEL



2) THE ENTITY-RELATIONSHIP (ER) MODEL

- To create a Relational Database, the traditional way is to use the **Entity-Relationship (ER) Model** which has three levels:
 - The **conceptual** model: a diagram representing the entities and their relationships
 - The **logical** model: a list of tables
 - The **physical** model: the organization of the information on the computer hard drive (files)



3) THE ENTITY-RELATIONSHIP (ER) MODEL

- Allows you to draw the **Entity-relationship Diagram (ERD)** which is the **database design**
- Is an easy-to-use graphical tool for **modeling data**
- Widely used in database design
- Often done first with **iterations** to make perfect
- Goes from **conceptual** to **logical** to **physical**

Source: Jasmin Jakupovic “Databases” course at JU

3) THE ENTITY-RELATIONSHIP (ER) MODEL

- Contextual data:
 - To set something in context, to further explain, and to give more value: sometimes you need context to a story.

Zoe, Foe, 21, F, 070-123 456

- Explained...

Data value	Context
Zoe	First Name
Foe	Last Name
21	Age
F	Gender
070-123 456	Phone

3) THE ENTITY-RELATIONSHIP (ER) MODEL

From data to database

Data value	Context
Zoe	First Name
Foe	Last Name
21	Age
F	Gender
070-123 456	Phone

Database

fName	IName	Age	Gender	Phone
Zoe	Foe	21	F	070-123 456
Erik	Doe	26	M	071-236 456

Entity: Person

- Context became column (attribute)
- Data value became row (tuple)
- Entity is person
- An 'entity' is a table!

3) THE ENTITY-RELATIONSHIP (ER) MODEL

- Components of the **ER Diagram**:
 - **Entities**
 - **Attributes**
 - **Relationships**
 - **Cardinalities**

3.1) THE CONCEPTUAL MODEL

- Notation, how to visualize it!

Entity

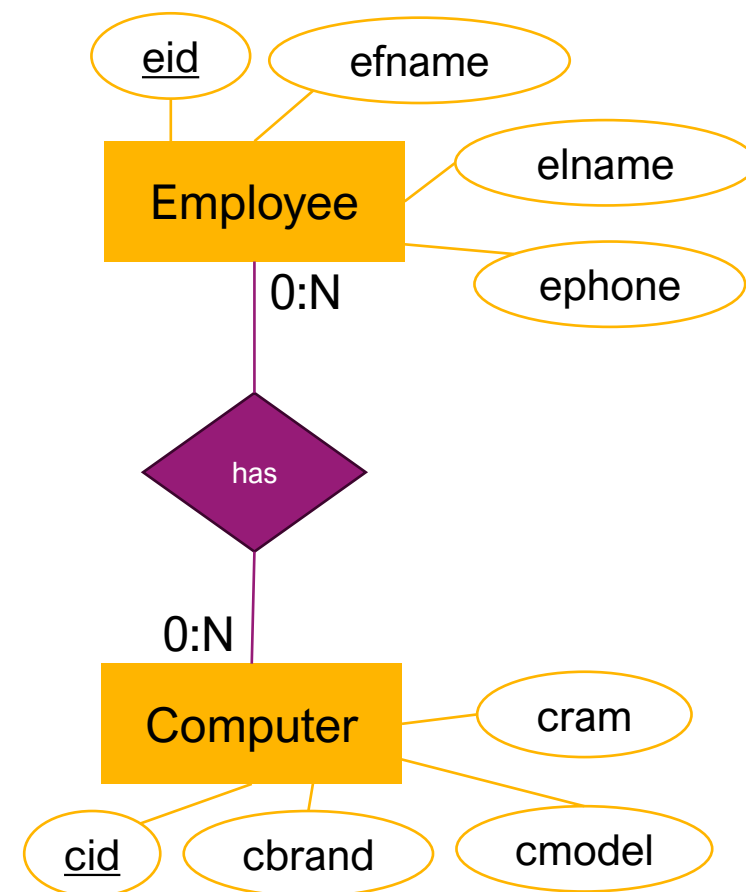
Concept, object, place, event from the real-world, a **noun**: ['Person', 'Car', 'Place', 'Course', 'Account'...]

attribute

Attribute, property or characteristic linked to an entity, a **noun**: ['id', 'firstName', 'lastName', 'dateOfBirth', 'city', 'phone', 'email', 'brand', 'model'...]

relation

Association between at least two entities, a **verb**: ['has', 'is', 'belongs', 'contains', 'owns'...]

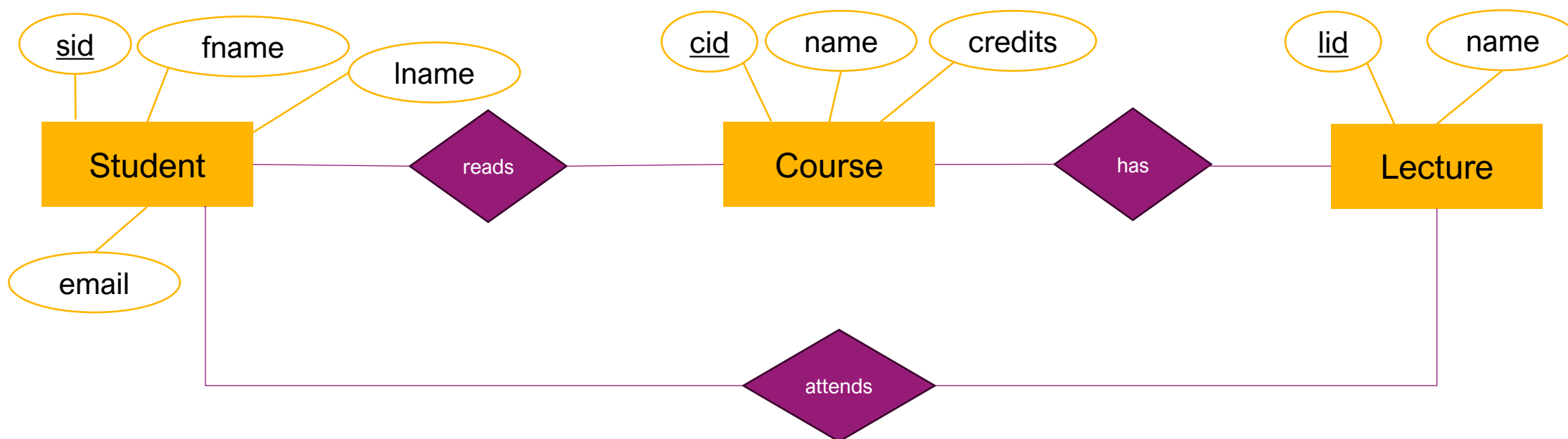


3.1) THE CONCEPTUAL MODEL

- Example: A university database identifies Students, Courses and Lectures as entities. The student has attributes such as ID number, Name, E-mail, Programme. A student usually reads a course which has a number of lectures.

3.1) THE CONCEPTUAL MODEL

- Example: A university database identifies Students, Courses and Lectures as entities. The student has attributes such as ID number, Name, E-mail, Programme. A student usually reads a course which has a number of lectures.



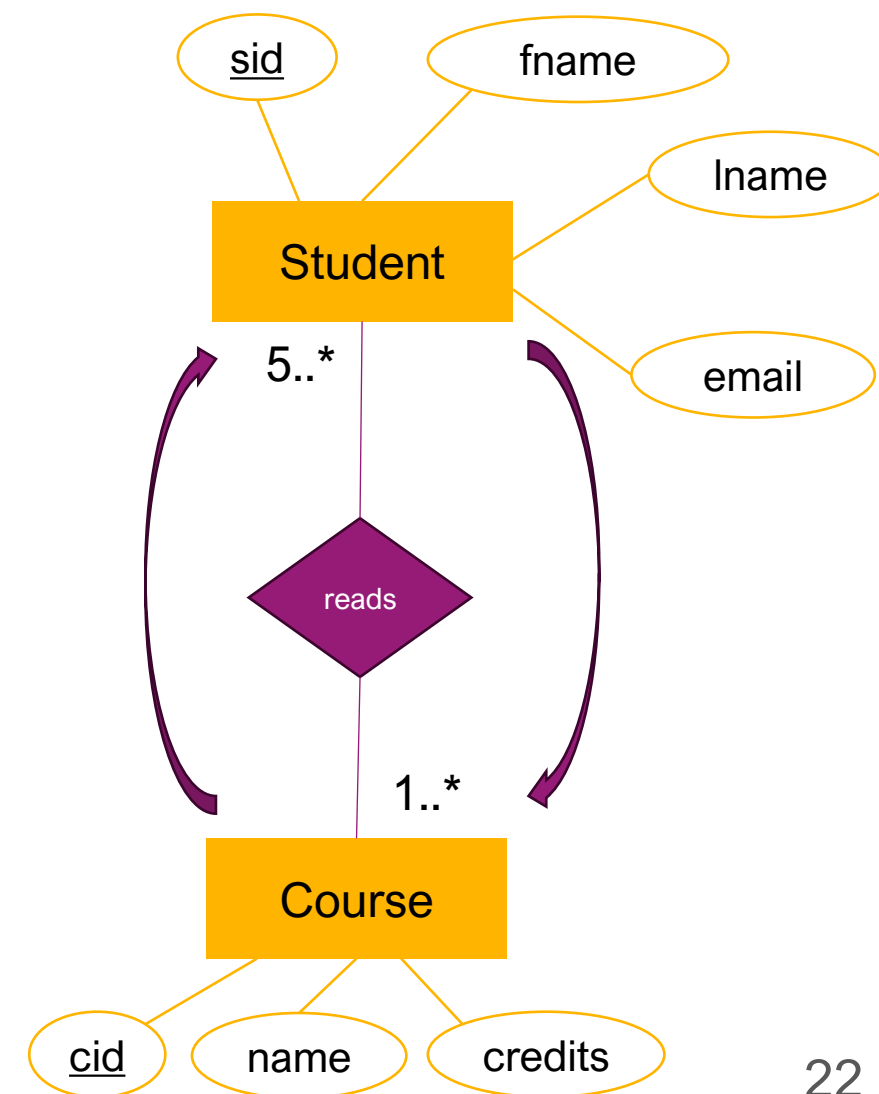
Conceptual / Entity-Relationship Diagram (ERD)

3.1) THE CONCEPTUAL MODEL

- Cardinality:
 - It is a key component of every ER diagram. The cardinality defines the numerical attributes (min..max) of the relationship between two entities or entity sets.

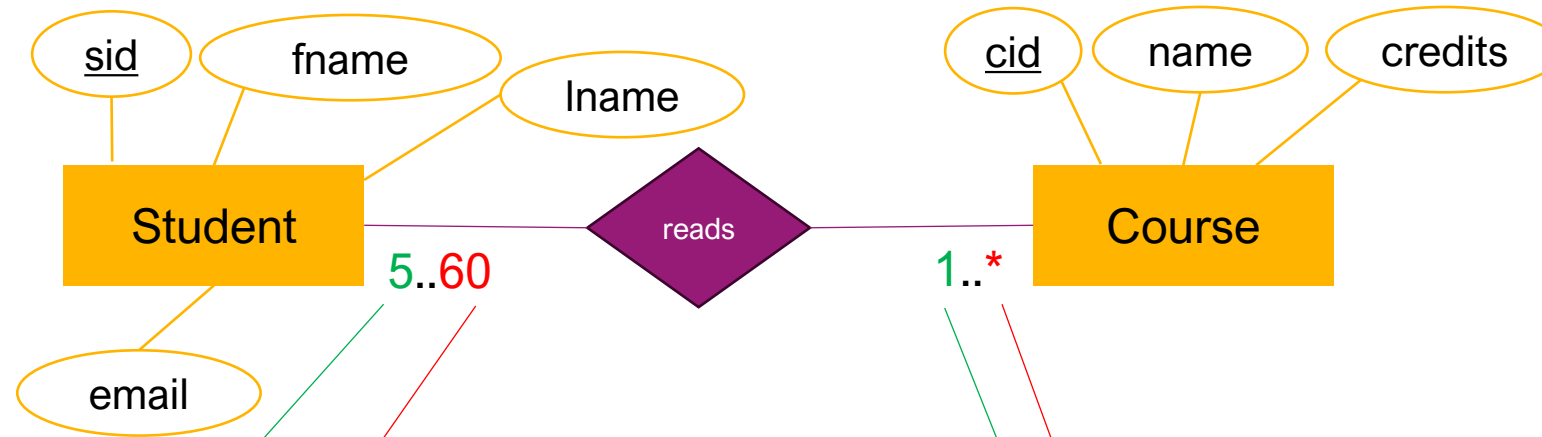
Q1: How many courses does a student have to read?

Q2: How many students does a course have?



3.1) THE CONCEPTUAL MODEL

- Cardinality explained:



min..max

A course is read by at least 5 students (minimum)

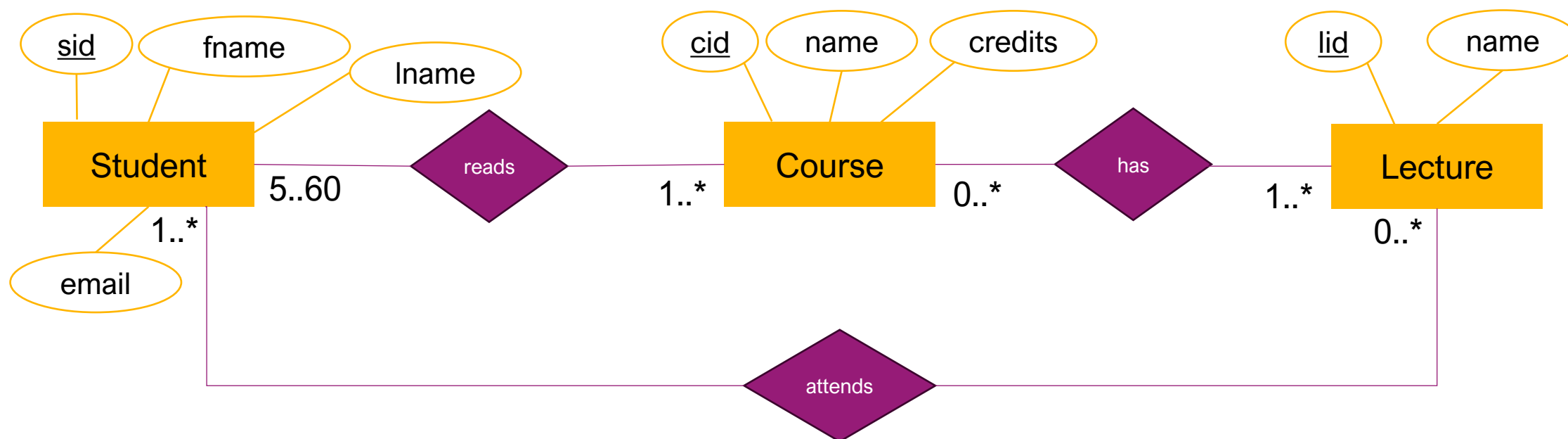
A course is read by maximum 60 students (maximum)

A student can read at least 1 course (minimum)

A student can read many courses (maximum)

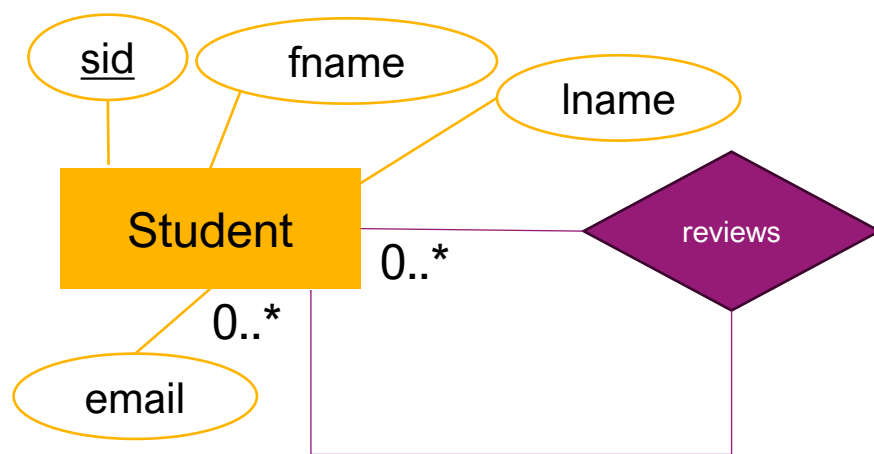
3.1) THE CONCEPTUAL MODEL

- Final conceptual ER diagram with cardinalities:



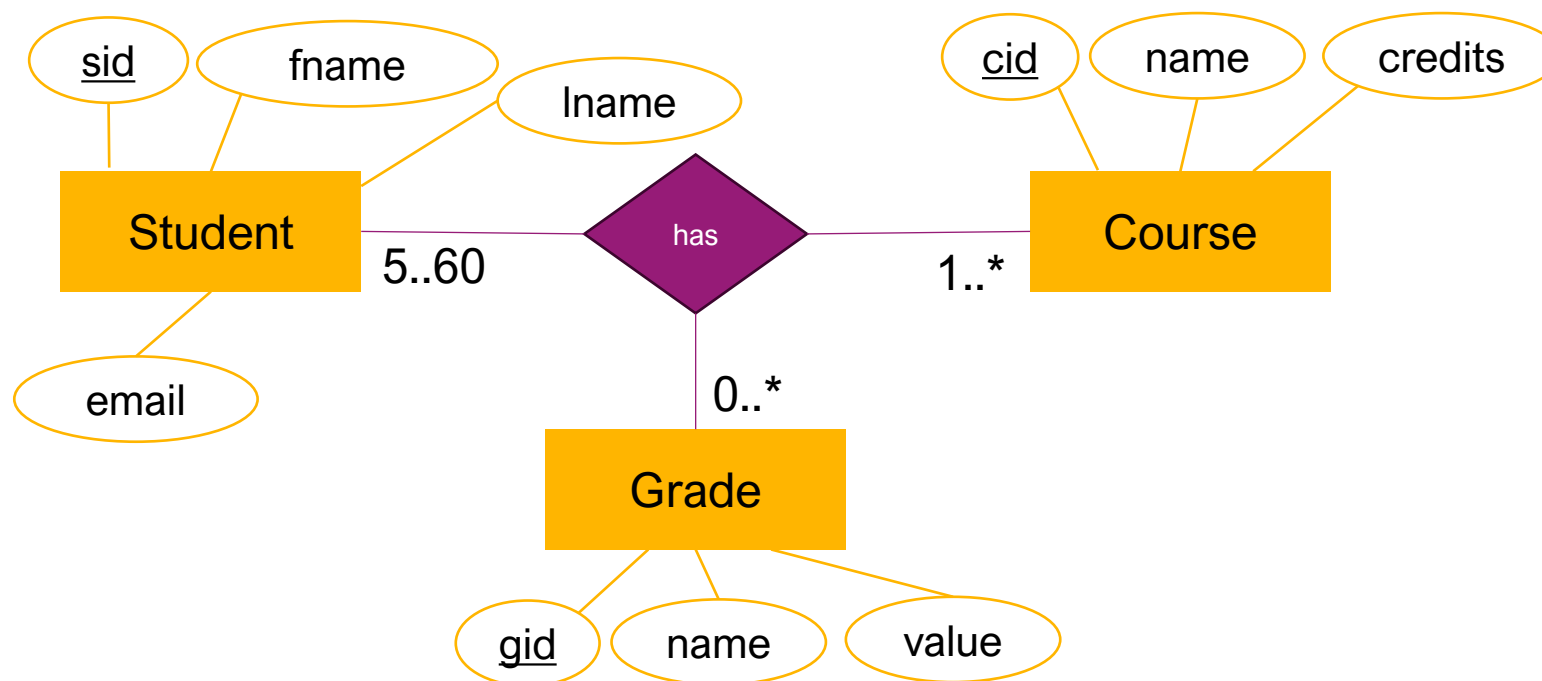
3.1) THE CONCEPTUAL MODEL

- Recursive relations are allowed:



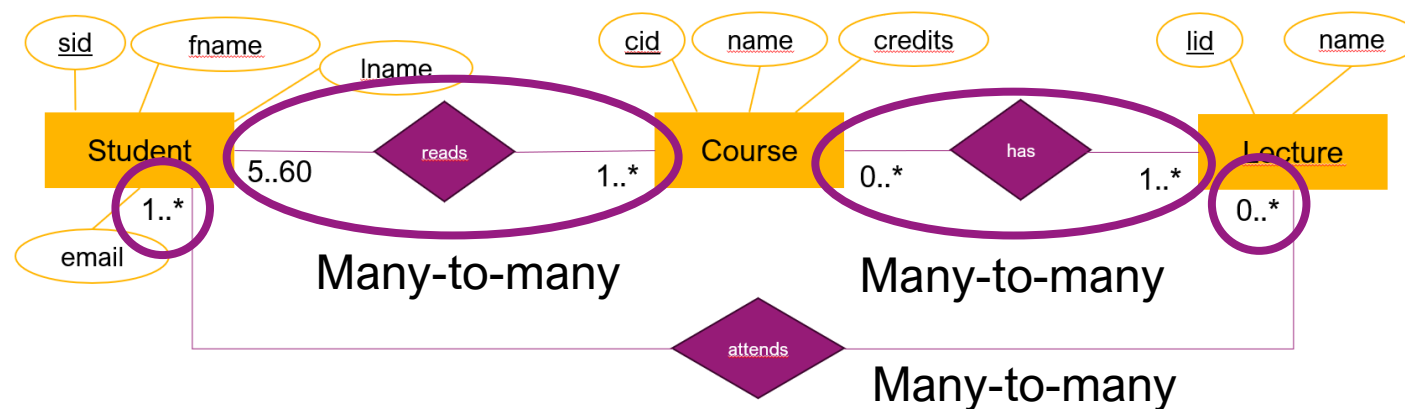
3.1) THE CONCEPTUAL MODEL

- Relations with more than one entities are allowed:



3.1) THE CONCEPTUAL MODEL

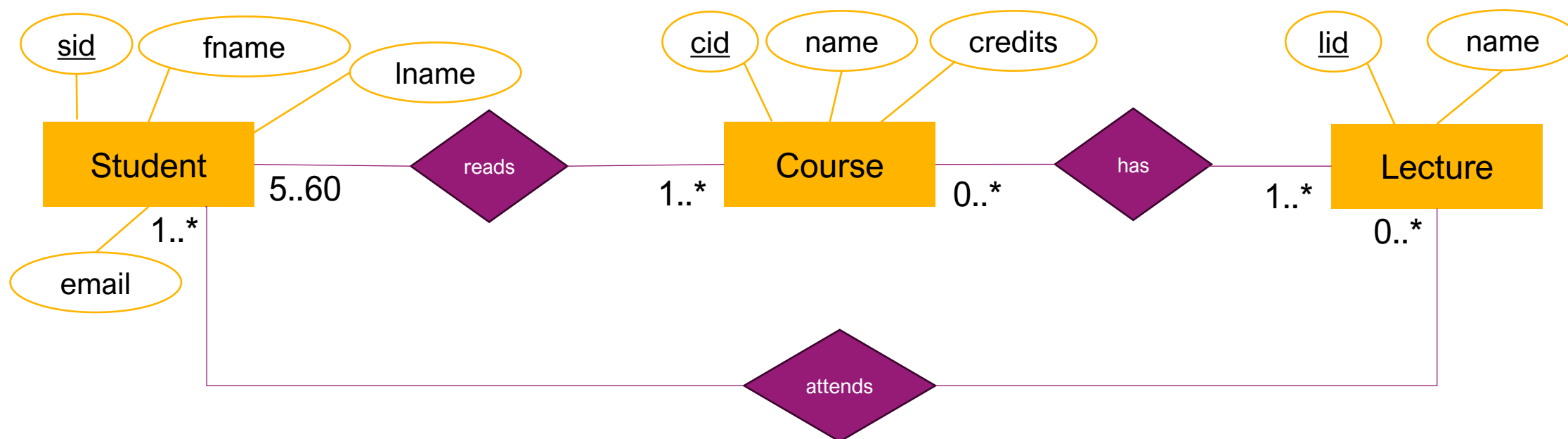
- There can have three kinds of relationship in a conceptual model:
 - One-to-one
 - One-to-many
 - Many-to-many



- To **identify** them, consider only the **MAX** values of the cardinalities

3.2) THE LOGICAL MODEL

- How to go from this drawing to the logical model?



3.2) THE LOGICAL MODEL

- You just follow simple rules:

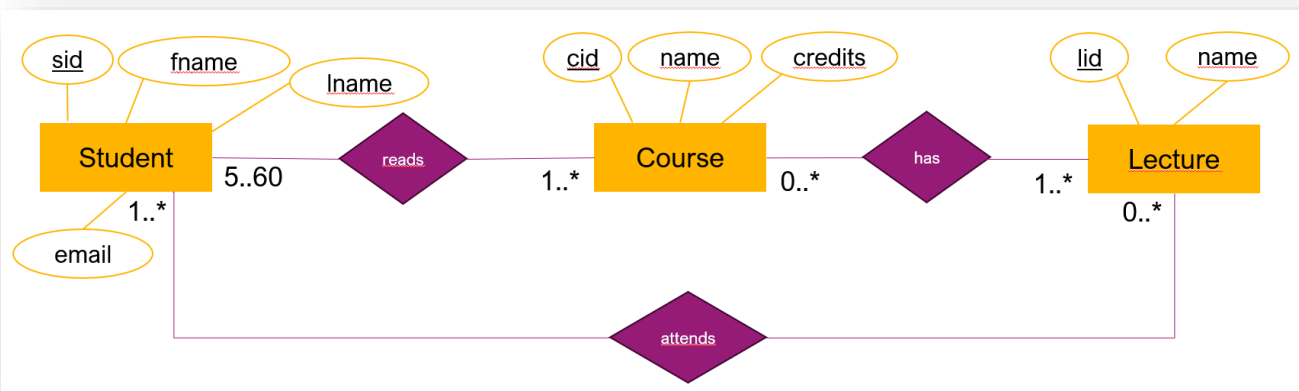
RULES:

- 1) Any **entity** becomes a **table**.
- 2) **Look** at the **cardinalities (max only)**:
 - a. a **one-to-many** cardinality makes the **PK** on the 1 side become a **FK** into the table on the * side.
 - b. a **many-to-many** relation becomes a **full table** with **PK** being the two **FK** of the tables in the relation.

- That is why the **cardinalities** are **SO IMPORTANT!**

3.2) THE LOGICAL MODEL

- Let's do it!



RULES:

- Any **entity** becomes a **table**.
- Look at the **cardinalities (max only)**:
 - a **one-to-many** cardinality makes the **PK** on the 1 side become a **FK** into the table on the * side.
 - a **many-to-many** relation becomes a **full table** with **PK** being the two **FK** of the tables in the relation.

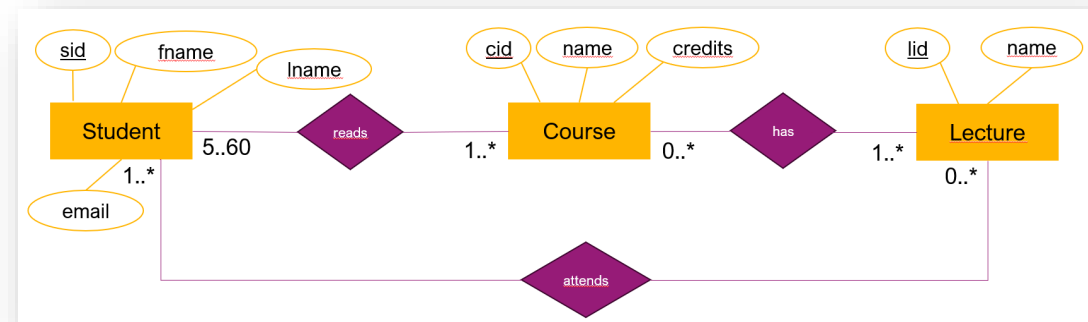
1) 3 Entities (rectangles) => minimum **3 tables**

2)b. 3 Many-to-many cardinalities (min:max) => **3 more tables**

=> **6 tables** in the database

3.2) THE LOGICAL MODEL

- TABLES:



3.3) THE PHYSICAL MODEL

- This model deals with the data organization on your hard drive: it is **PHYSICAL**
- How many **files** do you need?
- Do you need an **index** to speed up the search for data?
- How they will be organized in the same **directory**, in multiple ones, on the same **server**, on the **cloud**?...

3) THE ENTITY-RELATIONSHIP (ER) MODEL

- Once you have done the two (over three) steps: conceptual and logical,
- Congratulations, you have modeled your real-life problem into a computer database!
- Now, we need a **language** to **describe our data** and **interact** with our database

SQL license plate should appear here,
but it does not exist in Sweden: no Q in car plates! ☹️

3) THE ENTITY-RELATIONSHIP (ER) MODEL

- Once you have done the two (over three) steps: conceptual and logical,
- Congratulations, you have modeled your real-life problem into a computer database!
- Now, we need a **language** to **describe our data** and **interact** with our database



3) THE ENTITY-RELATIONSHIP (ER) MODEL

- Once you have done the two (over three) steps: conceptual and logical,
- Congratulations, you have modeled your real-life problem into a computer database!
- Now, we need a **language** to **describe our data** and **interact** with our database



3) THE ENTITY-RELATIONSHIP (ER) MODEL

- Once you have done the two (over three) steps: conceptual and logical,
- Congratulations, you have modeled your real-life problem into a computer database!
- Now, we need a **language** to **describe our data** and **interact** with our database



4) MS STRUCTURED QUERY LANGUAGE (SQL)

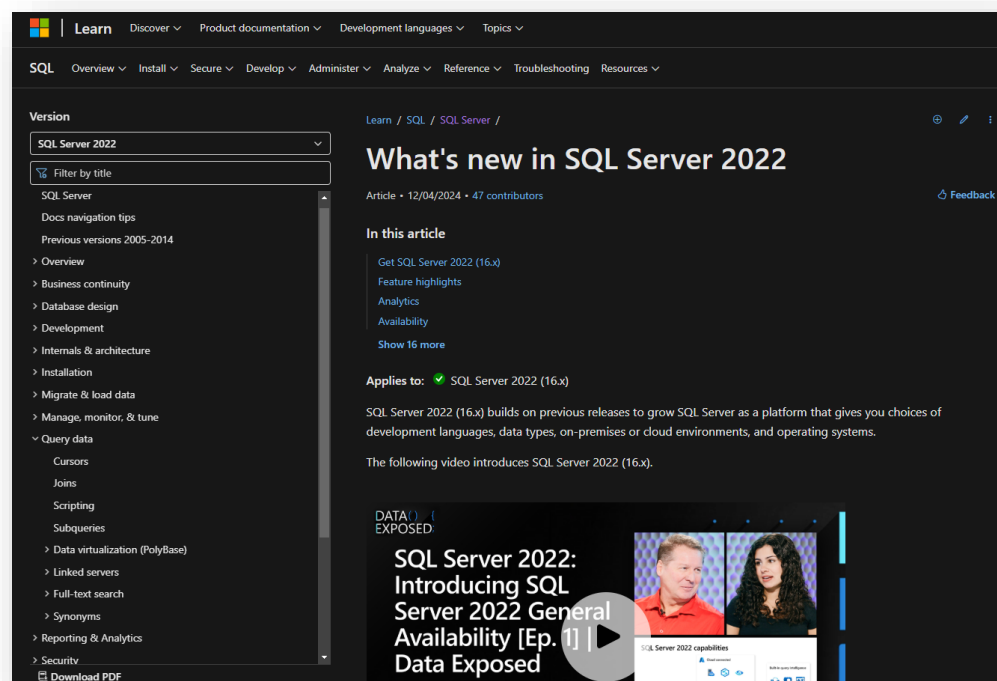
- SQL is a language to **Create, Read, Update, and Delete** data from a relational database.

C R U D

- SQL is a "standard" language that works (with several syntax changes) with ALL the different relational databases systems

4) MS STRUCTURED QUERY LANGUAGE (SQL)

- In this course, we will focus on the **MS SQL Server syntax** but, it can be adapted easily to other DBMS such as MySQL, Oracle, SQLite3...



4) MS STRUCTURED QUERY LANGUAGE (SQL)

- You can read the "**Datatypes**" page to get more information on the types of data you can use in MS SQL Server

Exact numerics

- tinyint
- smallint
- int
- bigint
- bit ¹
- decimal ²
- numeric ²
- money
- smallmoney

¹ The **bit** data type is used for storing Boolean values.

² The **decimal** and **numeric** data types are identical.

Approximate numerics

- float
- real

Date and time

- date
- time
- datetime2
- datetimeoffset
- datetime
- smalldatetime

Character strings

- char
- varchar
- text

4) MS STRUCTURED QUERY LANGUAGE (SQL)

- The MS SQL Server **syntax** is described on the official site

Version

SQL Server 2022

Filter by title

- > Data types
- > XML
- > DBCC
- > Functions
- > Language elements
- > Queries
 - Queries
 - > General
 - > SELECT
 - SELECT
 - SELECT Clause
 - SELECT Examples
 - SELECT - FOR Clause
 - SELECT - GROUP BY
 - SELECT - WINDOW Clause
 - SELECT - HAVING
 - SELECT - INTO Clause
 - SELECT - ORDER BY Clause
 - SELECT - OVER Clause
 - > FROM plus JOIN, APPLY, PIVOT
 - > WHERE
 - > Hints
 - > Predicates
 - > Statements
 - > xQuery

Download PDF

A. Use SELECT to retrieve rows and columns

The following example shows three code examples. This first code example returns all rows (no WHERE clause is specified) and all columns (using the *) from the `Product` table in the `AdventureWorks2022` database.

```
SQL
USE AdventureWorks2022;
GO

SELECT *
FROM Production.Product
ORDER BY Name ASC;

-- Alternate way.
USE AdventureWorks2022;
GO

SELECT p.*
FROM Production.Product AS p
ORDER BY Name ASC;
GO
```

This example returns all rows (no WHERE clause is specified), and only a subset of the columns (`Name`, `ProductNumber`, `ListPrice`) from the `Product` table in the `AdventureWorks2022` database. Additionally, a column heading is added.

```
SQL
USE AdventureWorks2022;
GO

SELECT Name,
       ProductNumber,
       ListPrice AS Price
FROM Production.Product
ORDER BY Name ASC;
GO
```

This example returns only the rows for `Product` that have a product line of `R` and that have days to manufacture that is less than `4`.

```
SQL
```


5) MARIADB SQL

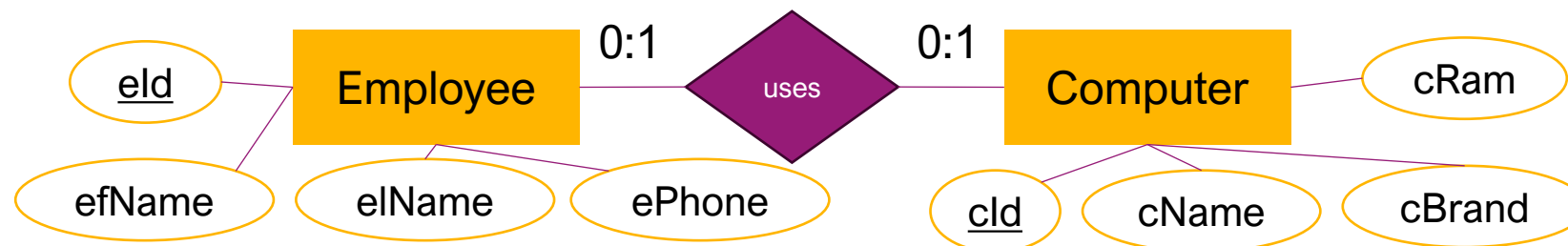
More information: https://mariadb.com/wp-content/uploads/2021/08/mariadb-standard-developer_cheat-sheet_1113.pdf

- **C**reate:
 - Create a table: **CREATE TABLE ...**
 - Create new data in a table: **INSERT INTO ... VALUES (...)**
- **R**ead: **SELECT * FROM ...**
- **U**ppdate:
 - Update a table content: **ALTER TABLE ... (Be careful)**
 - **UPDATE ... SET ... WHERE ...**
- **D**elete:
 - Delete a table: **DROP TABLE ... (Be careful)**
 - Delete one or more records (rows) in a table: **DELETE FROM ... WHERE ...**

6) STANDARD RELATIONSHIPS

- There are 3 standard relationships and 1 recursive one:
 - 1) One-to-One (1:1)
 - 2) One-to-Many (1:N) and Many-to-One (1:N)
 - 3) Many-to-Many (N:M)
 - 4) Recursive relationships

6.1) ONE-TO-ONE RELATIONSHIPS



Employee

<u>eld</u>	efName	eName	ePhone	cld
123	Jérôme	LANDRÉ	0708091011	33
241	Mira	POP	0607080910	27
324	Jasmin	JAKUPOVIC	0606060505	NULL

Primary Key
PK

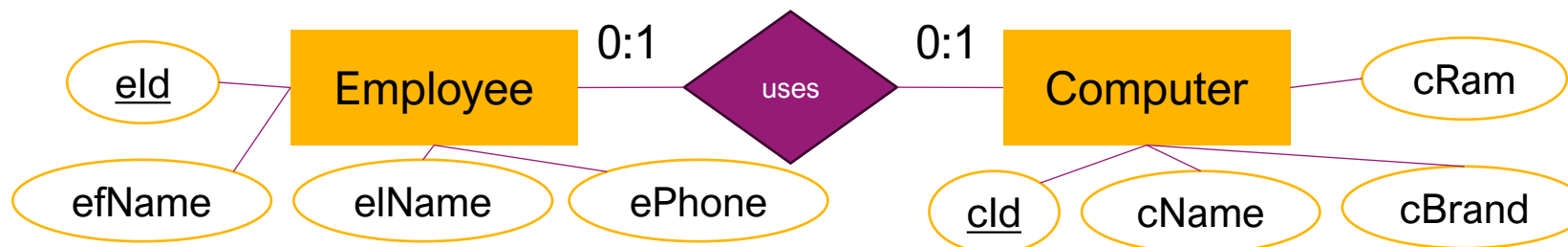
Foreign Key
FK

Computer

<u>cld</u>	cName	cBrand	cRam
33	laptopLJ	HP	16GB
124	serverCSI	IBM	64GB
27	laptopMP	HP	24GB

Primary Key
PK

6.1) ONE-TO-ONE RELATIONSHIPS



Computer

<u>cld</u>	cName	cBrand	cRam	eld
33	laptopLJ	HP	16GB	123
124	serverCSI	IBM	64GB	NULL
27	laptopMP	HP	24GB	241

Primary Key
PK

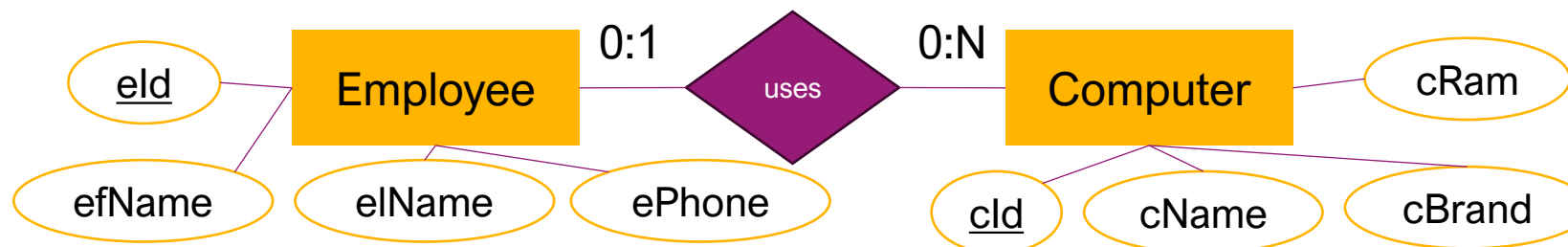
Employee

<u>eld</u>	efName	eName	ePhone
123	Jérôme	LANDRÉ	0708091011
241	Mira	POP	0607080910
324	Jasmin	JAKUPOVIC	0606060505

Primary Key
PK

Foreign Key
FK

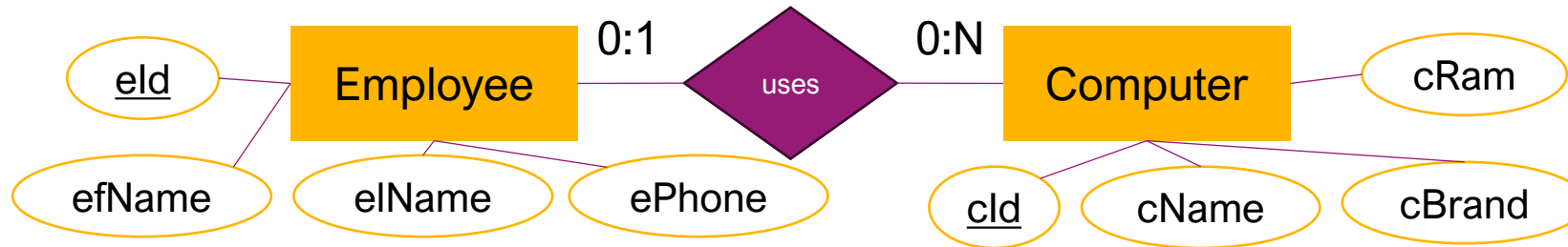
6.2) ONE-TO-MANY/MANY-TO-ONE RELATIONSHIPS



RULES:

- 1) Any **entity** becomes a **table**.
- 2) **Look** at the **cardinalities (max only)**:
 - a. a **one-to-many** cardinality makes the **PK** on the 1 side become a **FK** into the table on the * side.
 - b. a **many-to-many** relation becomes a **full table** with **PK** being the two **FK** of the tables in the relation.

6.2) ONE-TO-MANY/MANY-TO-ONE RELATIONSHIPS



Computer

<u>cld</u>	cName	cBrand	cRam	eld
33	laptopLJ	HP	16GB	123
124	serverCSI	IBM	64GB	NULL
27	laptopMP	HP	24GB	241
185	desktop33	HP	16GB	241

Primary Key
PK

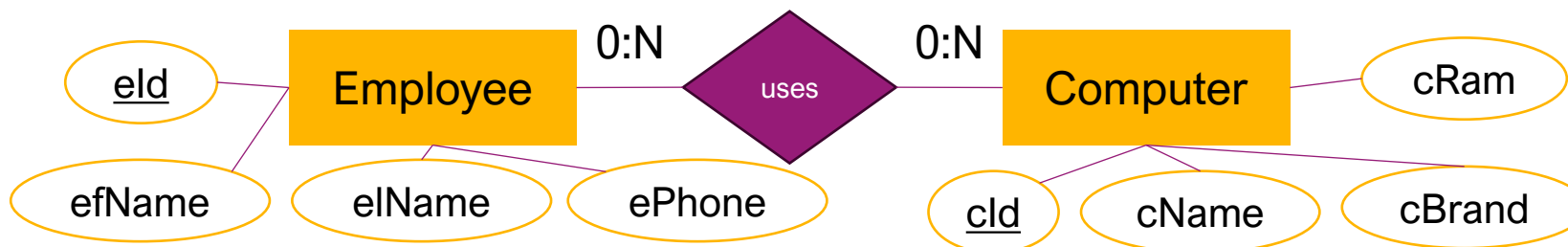
Employee

<u>eld</u>	efName	eName	ePhone
123	Jérôme	LANDRÉ	0708091011
241	Mira	POP	0607080910
324	Jasmin	JAKUPOVIC	0606060505

Primary Key
PK

Foreign Key
FK

6.3) MANY-TO-MANY RELATIONSHIPS



Employee

<u>eld</u>	efName	eName	ePhone
123	Jérôme	LANDRÉ	0708091011
241	Mira	POP	0607080910
324	Jasmin	JAKUPOVIC	0606060505

Primary Key
PK

EmployeeUsesComputer

<u>eld</u>	<u>cld</u>
123	33
241	27
241	185
123	185

eID: **FK** cld: **FK**
(eld, cld): **PK**

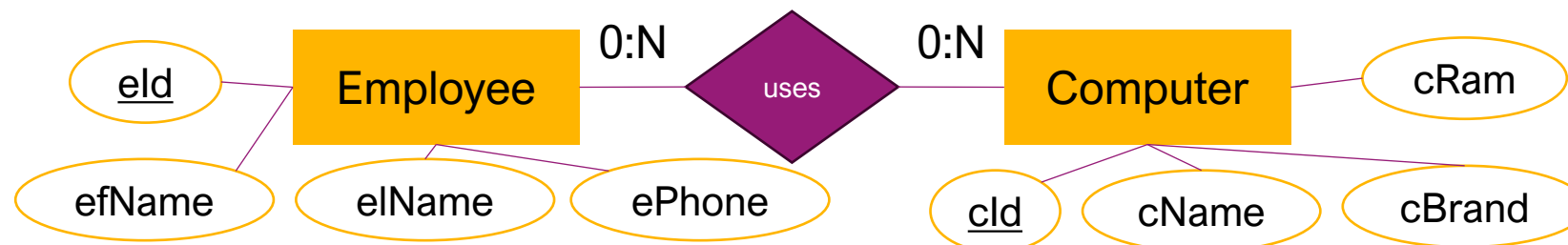
Computer

<u>cld</u>	cName	cBrand	cRam
33	laptopLJ	HP	16GB
124	serverCSI	IBM	64GB
27	laptopMP	HP	24GB
185	desktop33	HP	16GB

Primary Key
PK

Many-To-Many: We **MUST** have a middle-table or middle-relation!

6.3) MANY-TO-MANY RELATIONSHIPS



Employee

<u>eld</u>	efName	eName	ePhone
123	Jérôme	LANDRÉ	0708091011
241	Mira	POP	0607080910
324	Jasmin	JAKUPOVIC	0606060505

Primary Key
PK

EmployeeUsesComputer

<u>euclid</u>	eld	cld
1	123	33
2	241	27
3	241	185
4	123	185

eID: **FK** cld: **FK**

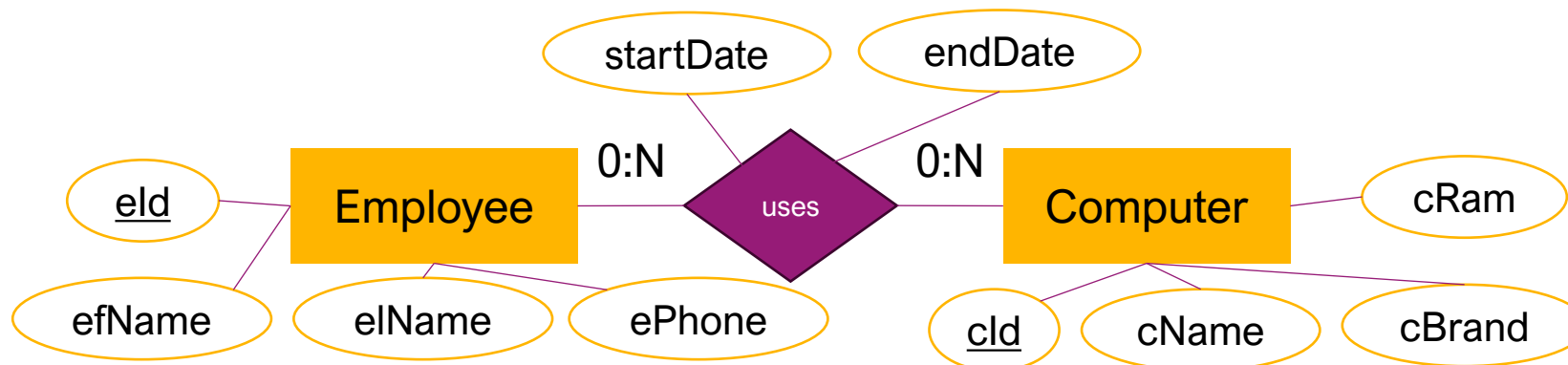
Primary Key
PK

Computer

<u>cld</u>	cName	cBrand	cRam
33	laptopLJ	HP	16GB
124	serverCSI	IBM	64GB
27	laptopMP	HP	24GB
185	desktop33	HP	16GB

Primary Key
PK

6.3) MANY-TO-MANY RELATIONSHIPS (WITH HISTORY)



Employee

<u>eld</u>	efName	elName	ePhone
123	Jérôme	LANDRÉ	0708091011
241	Mira	POP	0607080910
324	Jasmin	JAKUPOVIC	0606060505

Primary Key
PK

EmployeeUsesComputer

<u>eld</u>	<u>cld</u>	<u>startDate</u>	endDate
123	33	2025-01-01	NULL
241	27	2022-03-15	NULL
241	185	2024-03-21	NULL
123	185	2024-03-21	2025-01-13
123	185	2025-01-14	NULL

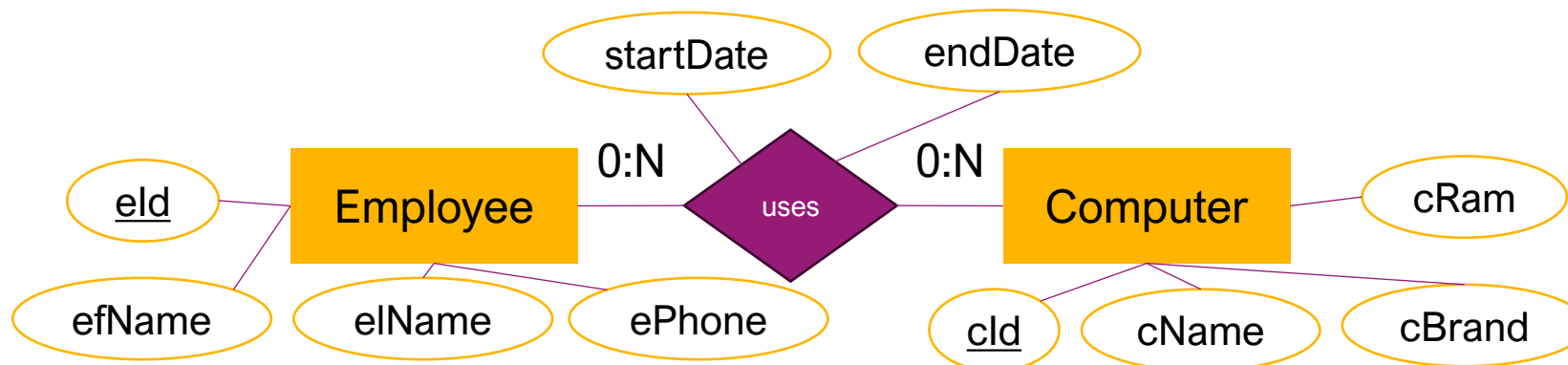
(eld, cld, startDate): **PK**

Computer

<u>cld</u>	cName	cBrand	cRam
33	laptopLJ	HP	16GB
124	serverCSI	IBM	64GB
27	laptopMP	HP	24GB
185	desktop33	HP	16GB

Primary Key
PK

6.3) MANY-TO-MANY RELATIONSHIPS (WITH HISTORY)



Employee

<u>eld</u>	efName	elName	ePhone
123	Jérôme	LANDRÉ	0708091011
241	Mira	POP	0607080910
324	Jasmin	JAKUPOVIC	0606060505

Primary Key
PK

EmployeeUsesComputer

<u>euclid</u>	eld	cld	startDate	endDate
1	123	33	2025-01-01	NULL
2	241	27	2022-03-15	NULL
3	241	185	2024-03-21	NULL
4	123	185	2024-03-21	2025-01-13
5	123	185	2025-01-14	NULL

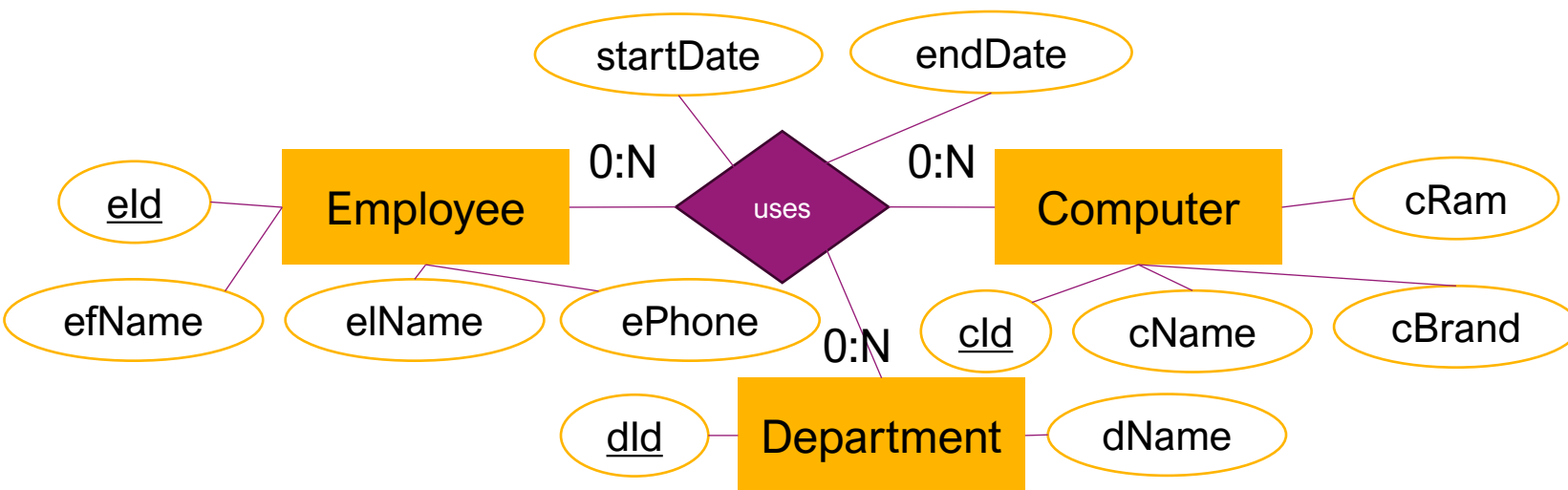
(euclid): **PK**

Computer

<u>cld</u>	cName	cBrand	cRam
33	laptopLJ	HP	16GB
124	serverCSI	IBM	64GB
27	laptopMP	HP	24GB
185	desktop33	HP	16GB

Primary Key
PK

6.3) MANY-TO-MANY RELATIONSHIPS (TERNARY RELATIONS)



Employee

<u>eld</u>	efName	eName	ePhone
123	Jérôme	LANDRÉ	0708091011
241	Mira	POP	0607080910
324	Jasmin	JAKUPOVIC	0606060505

EmployeeUsesComputerInDepartment

<u>euclid</u>	eld	cld	dld	startDate	endDate
1	123	33	117	2025-01-01	NULL
2	241	27	117	2022-03-15	NULL
3	241	185	27	2024-03-21	NULL
4	123	185	27	2024-03-21	2025-01-13
5	123	185	27	2025-01-14	NULL

(euclid): PK

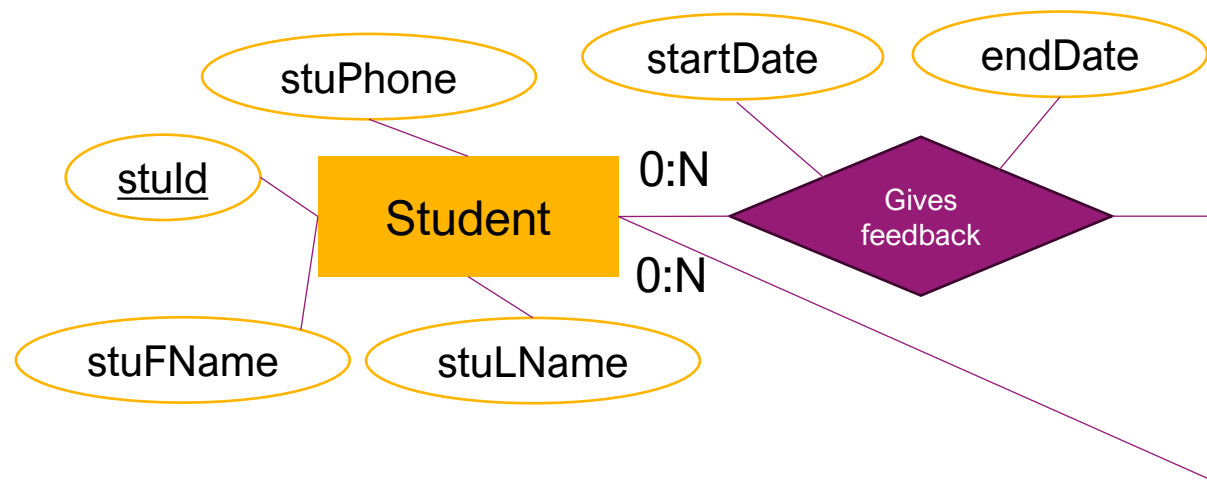
Department

<u>dld</u>	dName
117	Computer Science
124	Product Manufacturing
27	Math
185	Physics

Computer

<u>cld</u>	cName	cBrand	cRam
33	laptopLJ	HP	16GB
124	serverCSI	IBM	64GB
27	laptopMP	HP	24GB
185	desktop33	HP	16GB

6.3) MANY-TO-MANY RELATIONSHIPS (RECURSIVE RELATIONS)



Student

<u>stuId</u>	stuFName	stuLName	stuPhone
999	Maria	ERIKSSON	0605050505
1256	Sonia	LUNDBERG	0607080910
5600	Linus	LILLEBO	0600777006

StudentGivesFeedbackToStudent

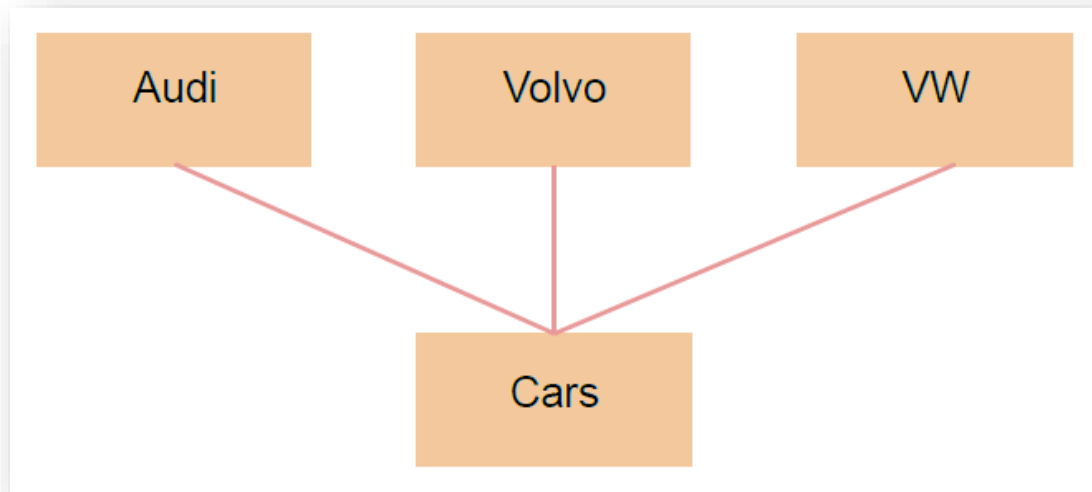
<u>sgfsId</u>	stuld1	stuld2	startDate	endDate
1	999	1256	2022-05-23	2022-06-17
2	999	5600	2024-03-15	NULL
3	5600	1256	2024-03-21	NULL
4	5600	99	2024-11-21	2025-01-13

(sgfsId): PK FK FK

7) SPECIAL RELATIONSHIPS

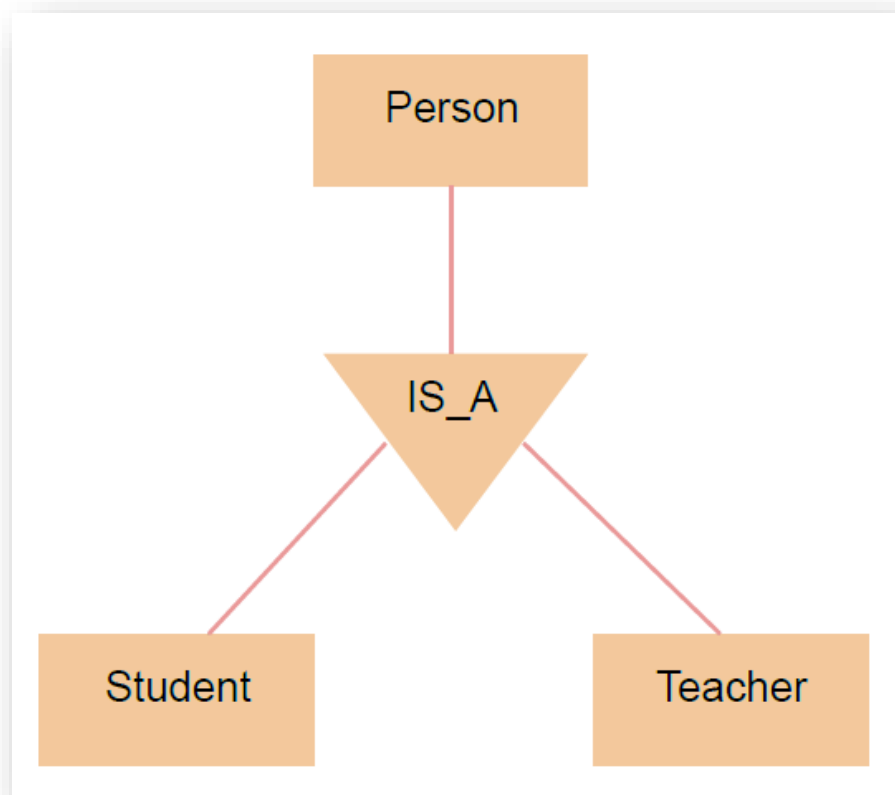
- There are special relationships (they are **similar** in practical use cases):
 - 1) Generalization
 - 2) Specialization
 - 3) Inheritance

7.1) GENERALIZATION



- Is the process of **generalizing** entities, where the generalized entities **contain** the properties of all the other entities.
- In generalization, a number of entities are **brought together** into one generalized entity based on their characteristics.

7.2) SPECIALIZATION (IS_A)

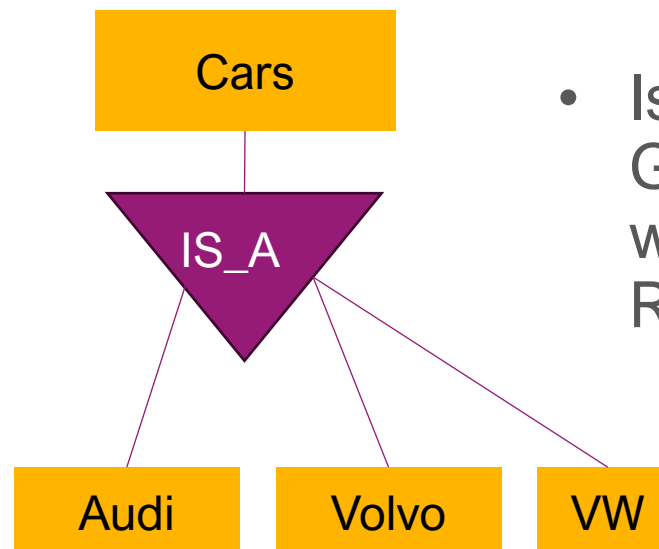
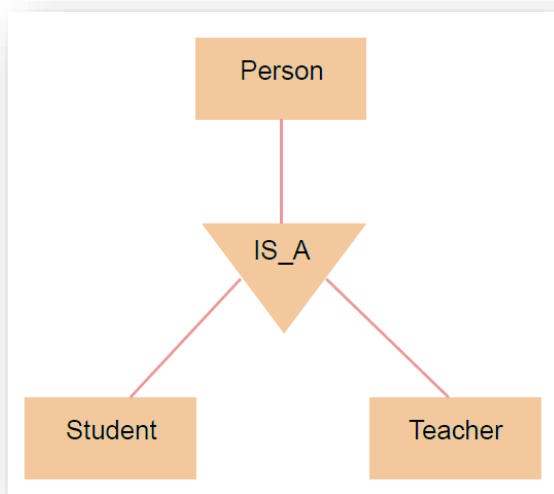
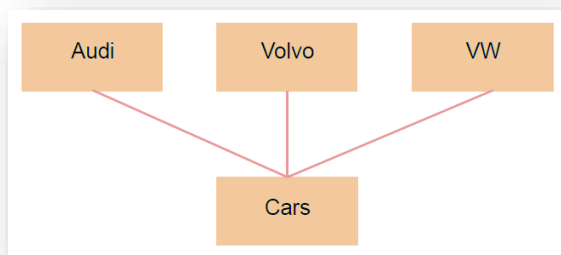


- Is the symmetric of **generalization**. Here, a group of entities is divided into sub-groups based on their characteristics.

e.g. "Person" has a first name, last name, birthDate, gender.

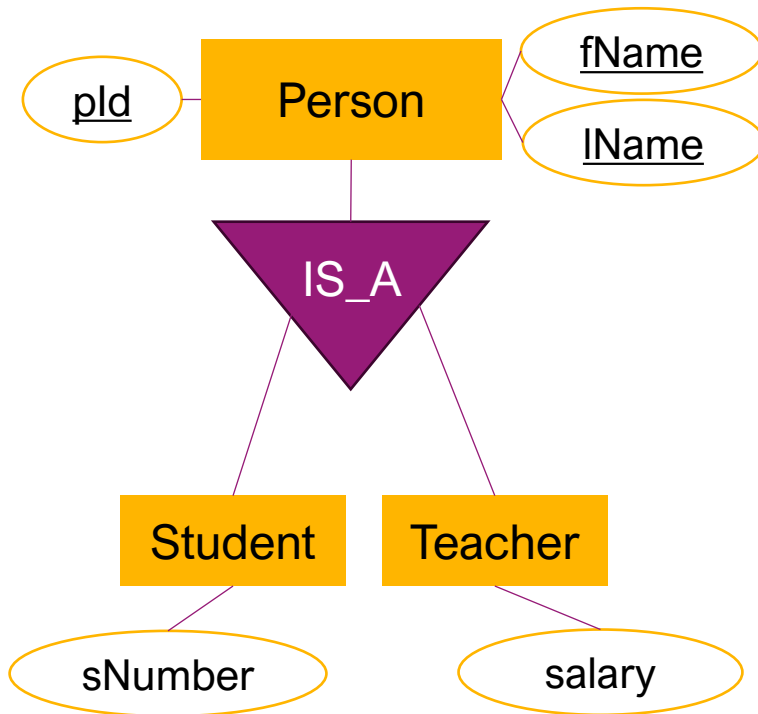
- These properties are common for all persons. But, in a company, persons can be identified as employee, employer, customer, vendor, teacher, student, based on the role in the company.

7.3) INHERITANCE



- Is the representation of Generalization and Specialization we will use in our Entity-Relationship Diagrams (ERD)

7.3) INHERITANCE



IS_A requires **special cardinalities**:

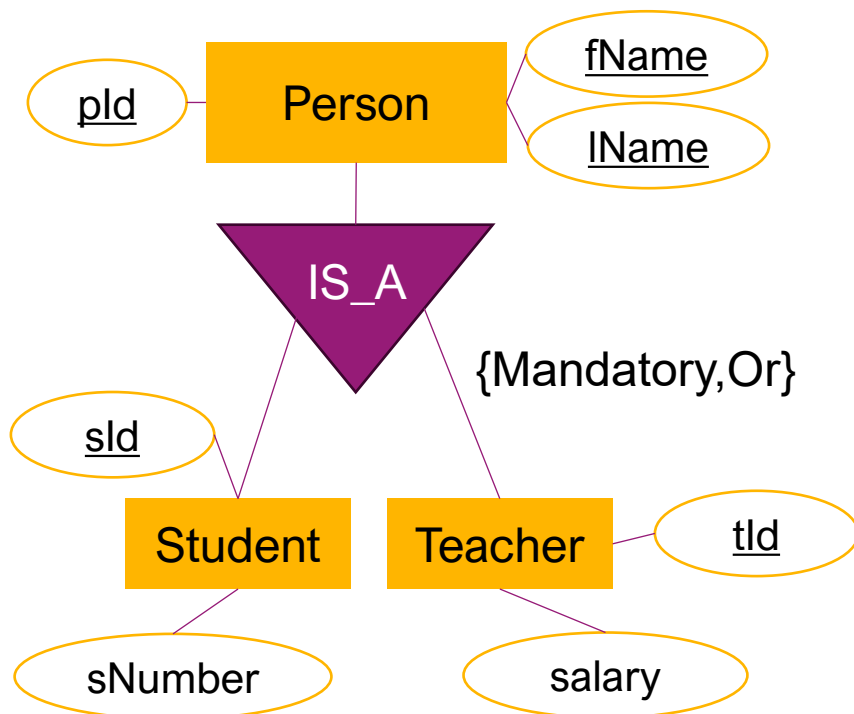
{Mandatory,And} - Must belong to a subclass and can belong to many

{Mandatory,Or} - Must belong to a subclass but only one subclass

{Optional,And} - Does not have to belong to a subclass but can belong to many

{Optional,Or} - Does not have to belong to a subclass but can belong to only one

7.3) INHERITANCE



Person

<u>pld</u>	fName	lName
123	Jérôme	LANDRÉ
241	Mira	POP
324	Jasmin	JAKUPOVIC
5678	Erik	ERIKSSON
6789	Robert	ROBERTSSON

PK

Student

<u>sld</u>	pld	sNumber
123	5678	2025-1234
241	6789	2025-1235

PK

FK

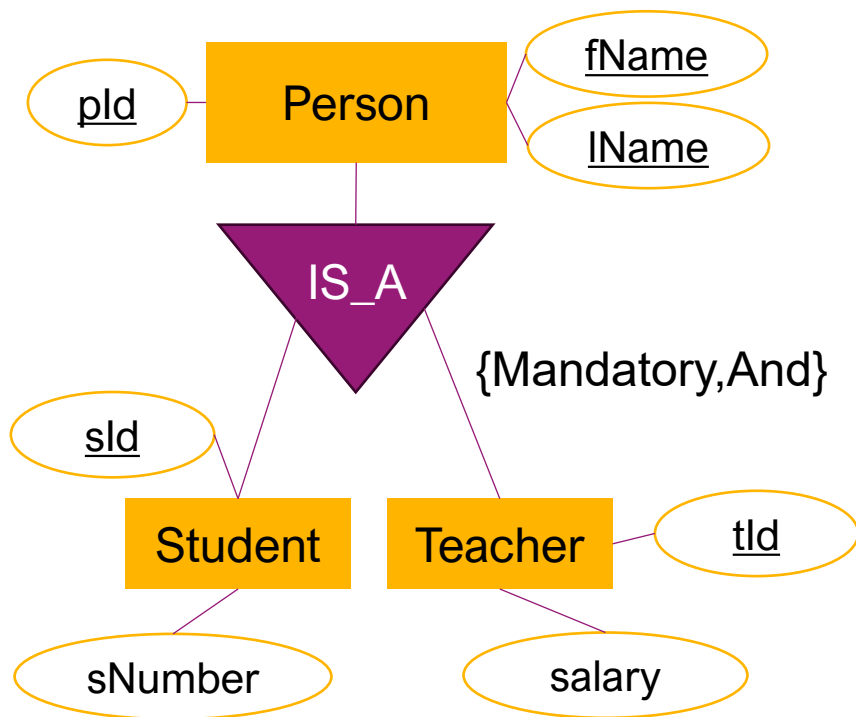
Teacher

<u>tld</u>	pld	salary
1	241	28000
2	123	32000
3	324	30000

PK

FK

7.3) INHERITANCE



A Teacher who is also a student

Person

<u>pld</u>	fName	lName
123	Jérôme	LANDRÉ
241	Mira	POP
324	Jasmin	JAKUPOVIC
5678	Erik	ERIKSSON
6789	Robert	ROBERTSSON

PK

Student

<u>sld</u>	pld	sNumber
123	5678	2025-1234
241	6789	2025-1235
1111	123	2022-9876

PK

FK

A Student who is also a teacher (lab assistant for instance)

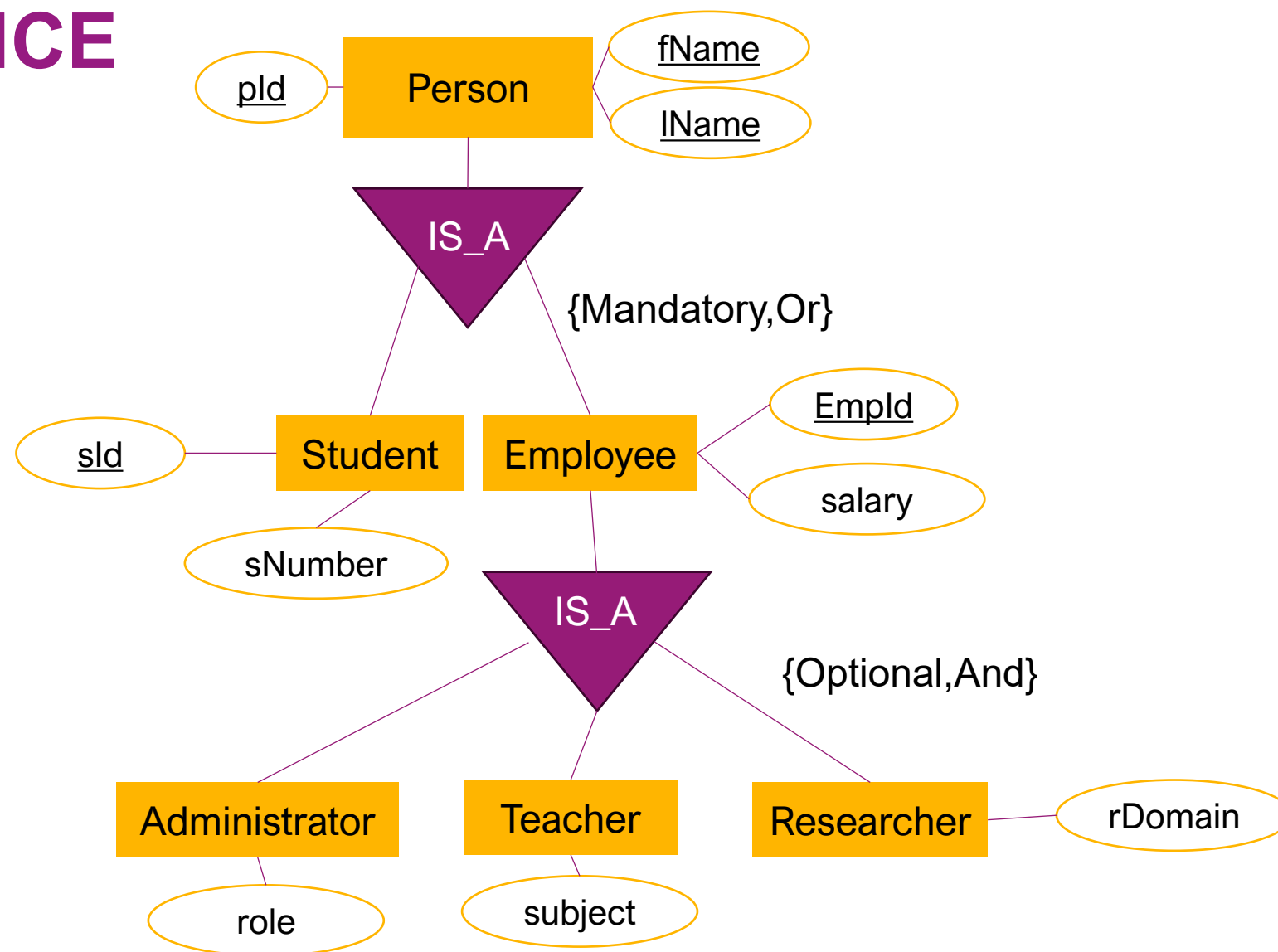
Teacher

<u>tld</u>	pld	salary
1	241	28000
2	123	32000
3	324	30000
4	6789	1700

PK

FK

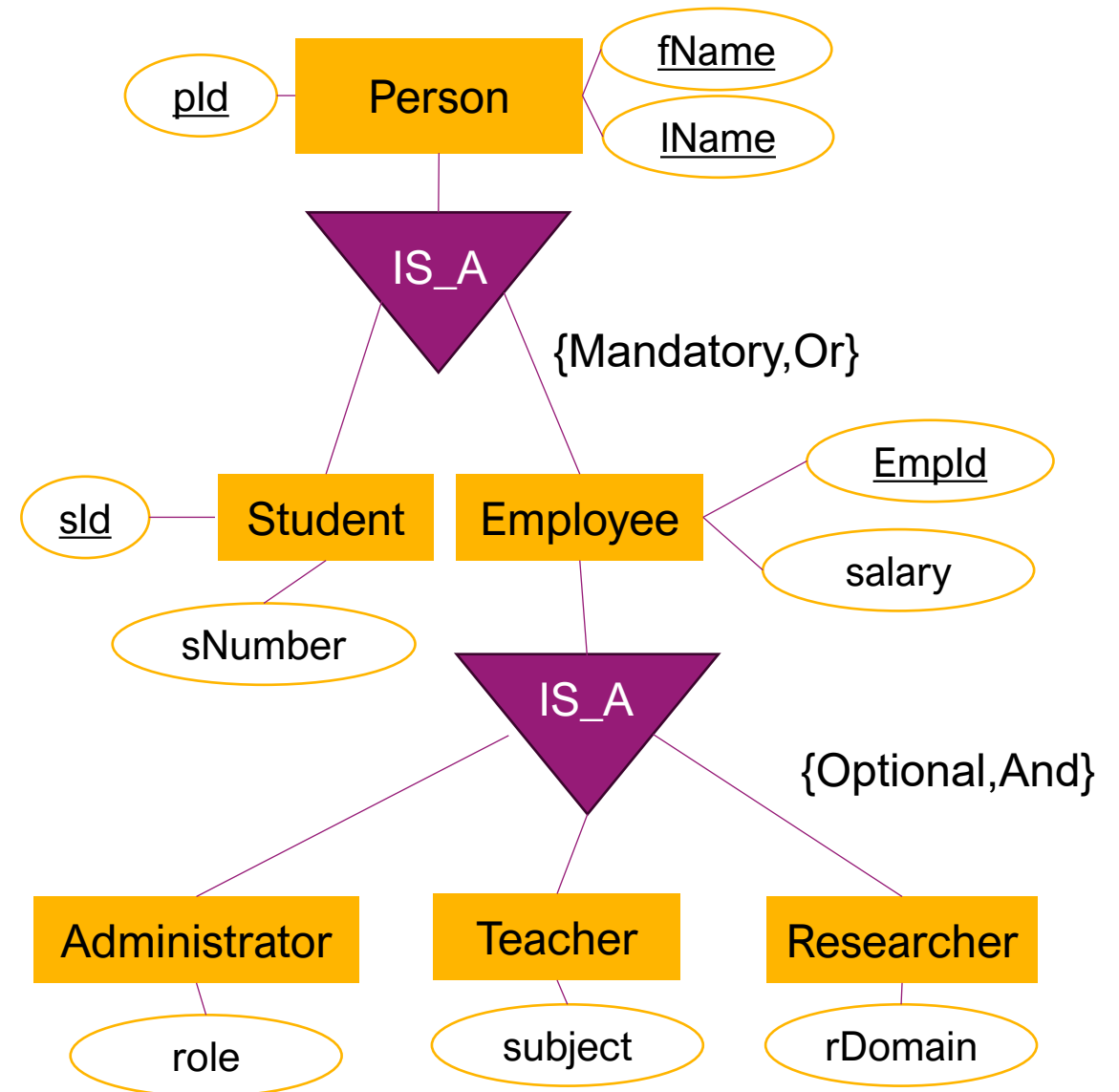
7.3) INHERITANCE



7.3) INHERITANCE

When to use?

- Must know rules when modelling
- Depending on scenario
- Used at conceptual level
- Implemented on logical level



IMPORTANT: REMEMBER TO BE POSITIVE!

- Yes, you can!



IMPORTANT: REMEMBER TO BE POSITIVE!

- Yes, you can!



ANY QUESTIONS?